

p5.penplotter

HOW A P5.JS SKETCH BECOMES A PLOT

Three methods on the p5 prototype, one object that draws and remembers at the same time, and a key that sends the last frame to the pen. This handbook explains what each method does, what it maps to in the engine, what it deliberately does not do, and where a sketch can go wrong.

Sebastien Vanblaere · 2026-09-30 · MIT · seb-prjcts-be.github.io/p5.penplotter · PDF of docs/handbook.html

Companion to the [vanilla.penplotter handbook](#), which covers the engine this adapter calls.

CONTENTS

1. The adapter
2. Installing: the version contract
3. Wiring a sketch: four ways and one timeline
4. createPlot(): one object for screen and paper
5. The shape methods, one by one
6. Fills
7. The frame model: sixty frames, one plot
8. plot.go(): connect, confirm, plot, stop
9. createPlotterEngine() and drawRoute()
10. What is not recorded
11. p5.waves in a sketch
12. The examples, one by one
13. Recipes
14. Pitfalls, with their error messages
15. Tests, tooling and the site
16. Limits and roadmap
17. Reference

1 THE ADAPTER

A p5 sketch runs sixty times a second and forgets every frame. A pen plotter draws once and keeps it. `p5.penplotter` is the smallest bridge between those two habits: you keep writing the sketch you always write, put `plot.` in front of the shapes that should reach paper, and press a key. The adapter adds three methods to the p5 prototype and nothing else:

METHOD	WHAT IT IS FOR
<code>createPlot(options)</code>	the one you will use: an object whose shape methods draw on the canvas <i>and</i> record the same shape in millimetres; <code>go()</code> sends it to the pen.
<code>createPlotterEngine(options)</code>	the bare engine with the canvas size as its page, when you want to talk to it directly.
<code>drawRoute(plan, options)</code>	draws a plan the way the machine will draw it: the real order, reversed paths, moved seams.

Everything else, geometry, optimisation, route planning, exports, machine profiles and the driver, lives in [vanilla.penplotter](#), which knows nothing about p5. The adapter contains no copy of any of it; a bug in the pipeline is fixed there, once, for both libraries. The one piece of real work the adapter does own is the translation from canvas pixels to millimetres on the bed.

It is not `p5.plotSvg`, which records a sketch between `beginRecordSvg()` and `endRecordSvg()` into an SVG file and covers far more of p5's drawing API. That library stops at the file on purpose; this one has no file at all. The two are not rivals. A `p5.plotSvg` file can be plotted by the engine's *SVG to pen* example.

2 INSTALLING: THE VERSION CONTRACT

Installation is explicit. The adapter is handed the p5 constructor and the engine class, and, when you want to plot, the engine's driver module:

```
import { PlotterEngine } from "https://cdn.jsdelivr.net/gh/seb-prjcts-be/vanilla.penplotter@v0.3.1/vanilla.penplotter.js";
import * as Ebb from "https://cdn.jsdelivr.net/gh/seb-prjcts-be/vanilla.penplotter@v0.3.1/src/driver/ebb.js";
import { installP5Penplotter } from "https://cdn.jsdelivr.net/gh/seb-prjcts-be/p5.penplotter@v0.2.1/p5.penplotter.js";

installP5Penplotter(p5, PlotterEngine, { driver: Ebb });
```

Why not auto-detect a global? Because load order and versions would become invisible, and the first thing that goes wrong with two libraries loaded from two places is a version mismatch nobody can see. So the contract is written down once, in code, as `REQUIRES = { core: "0.2.0", p5: "2.2.2" }`, and everything else is tested against it: `package.json`'s peer dependencies, the manifest, and the *Requirements* table in the README, which is byte for byte the same table as in the engine's README.

At install time the adapter checks three things. `p5` must be a constructor with a prototype. `PlotterEngine` must be a class. And when a driver is passed, it must expose `EbbDriver`, `createWebSerialTransport` and `EBB_PROFILES`, and the engine's static `version` must be 0.2.0 or newer, because an older engine has no driver at all. The refusal is a plain sentence: "p5.penplotter needs vanilla.penplotter 0.2.0 or newer for direct plotting; this one is 0.1.9. Update vanilla.penplotter first." Without a driver, nothing is checked and `createPlotterEngine()` and `drawRoute()` work with any engine; only `go()` will throw.

PINNED OR LATEST

The jsDelivr URLs above are pinned to a tag: a sketch that works today works next year. The GitHub Pages URLs (<https://seb-prjcts-be.github.io/.../>) always serve the latest `main` of both libraries, which is what the site's own examples use, with the version check as their guardrail. After an update, a browser can hold the old engine in its cache and the adapter will refuse it, correctly; reload with Ctrl+F5.

3 WIRING A SKETCH: FOUR WAYS AND ONE TIMELINE

Both libraries are ES modules, and a module has two properties that decide every setup: it creates no globals, so a `function setup()` written inside a module is invisible to p5's global mode, and it is deferred, so it runs after the page has been read but before the `load` event on which p5 starts a global-mode sketch. The timeline of any page is therefore:

- | | |
|-----------------------------------|-------------------------------------|
| 1. p5.js loads | (classic script, runs at once) |
| 2. classic scripts run in order | (your sketch.js defines setup/draw) |
| 3. page fully read | |
| 4. module scripts run | (the adapter is installed here) |
| 5. DOMContentLoaded | |
| 6. load → p5 calls window.setup() | (createPlot already exists) |

The four ways to place the three import lines on that timeline, in order of simplicity:

- 1. Global mode, wiring in index.html.** The imports live in one inline module in `index.html`; `sketch.js` is a plain script with no imports, exactly like every other p5 sketch. The one for classrooms and for the p5.js Web Editor. Rules: load p5 before the module; never give `sketch.js type="module"`; call `createPlot()` inside `setup()`, never at the top of the file, because at that moment the adapter is not installed yet.
- 2. Instance mode, sketch as a module.** The imports move into `sketch.js`, which hands itself to p5 with `new p5(...)` and prefixes every p5 call with `p.`. Self-contained and free of load-order assumptions; the form the library's own examples use for anything that imports.
- 3. Global mode from a module.** The sketch is a module and publishes its functions itself: `window.setup = setup;` and so on, one line per p5 callback you use. Forget one and p5 silently never calls it.
- 4. Import maps and local neighbours.** Not a fifth form but a refinement of any of the above: bare names like `"p5.penplotter"` resolved by an import map in `index.html`, or relative paths to sibling folders while working on the libraries themselves.

The [Setup](#) page shows all four in full, with what breaks and how the error reads. Whichever you pick, plotting for real needs the same three things: the driver handed in at install, Chrome or Edge on `localhost` or https (Web Serial exists nowhere else; not in Firefox, not in Safari, not in the Web Editor's iframe, not from a `file://` URL), and one click on the drawing, because a browser shows its port list only after a click; `plot.go()` itself is written at the end of the drawing and waits for that click.

4 CREATEPLOT(): ONE OBJECT FOR SCREEN AND PAPER

`createPlot(options)` returns a `P5Plot`. Its constructor decides one thing that everything else depends on: the scale between canvas pixels and millimetres on the bed.

```
plot = createPlot({ x: 147, y: 66, width: 300 });
// the canvas lands with its top-left corner 147 mm right and 66 mm down from the home
corner,
// and is 300 mm wide on the bed; the scale is 300 / width (of the canvas) mm per pixel
```

OPTION	MEANING	DEFAULT
<code>x</code> , <code>y</code>	where the canvas's top-left corner lands, in millimetres from the home corner	0, 0
<code>width</code>	how many millimetres wide the canvas is on the bed; sets the scale	none
<code>mmPerPixel</code>	the scale directly, when <code>width</code> is not given	1 (one millimetre per pixel)
<code>profile</code>	the engine's machine profile	<code>idraw-hse-a2</code>
<code>confirm</code>	a function that asks the user before plotting	<code>window.confirm</code>
<code>log</code>	a function that receives status messages	<code>console.log</code> with a <code>[plot]</code> prefix

Inside, the object holds a fresh `PlotterEngine` in millimetres. Its page is the profile's travel (594 × 432 mm) when a driver was installed, so the engine's bounds check is the machine's; without a driver it is just large enough to hold the canvas at its offset. Two small helpers do all the geometry: `toMm(x, y)` is `offset + pixel · scale`, and `toPx(point)` is its inverse, used to draw the engine's fills back onto the canvas.

`clear()` replaces the engine with an empty one. That is how a frame starts: every `draw()` that wants a fresh job calls `plot.clear()` first, exactly where it calls `background()`. `plan(options)` asks the engine for its plan and returns it; `engine` is the engine itself, for layers, pens and exports the adapter does not wrap.

5 THE SHAPE METHODS, ONE BY ONE

Every shape method does two things in one call: it calls the same p5 function with the same arguments, so the canvas shows what you wrote in the sketch's current style, and it records the shape in millimetres in the engine. The arguments are p5's, including its habit of taking a diameter where the engine takes a radius.

CALL	ON THE CANVAS	IN THE ENGINE
<code>point(x, y)</code>	<code>point</code>	a horizontal line of 0.24 mm: a pen cannot dot, it has to move to leave ink
<code>line(x1, y1, x2, y2)</code>	<code>line</code>	<code>line</code>
<code>circle(x, y, d)</code>	<code>circle</code>	<code>circle</code> with radius <code>d / 2 * scale</code> (a 96-gon)
<code>ellipse(x, y, w, h)</code>	<code>ellipse</code>	a circle when <code>w === h</code> ; otherwise a 96-point polygon sampled at $\pi/48$
<code>arc(x, y, w, h, start, stop, mode)</code>	<code>arc</code>	points every $\pi/48$ along the ellipse; <code>OPEN</code> (default) a polyline, <code>CHORD</code> a closed polygon, <code>PIE</code> a closed polygon through the centre; angles converted from degrees when <code>angleMode(DEGREES)</code>
<code>rect(x, y, w, h)</code>	<code>rect</code>	<code>rect</code> , corner mode
<code>square(x, y, s)</code>	<code>rect</code>	<code>rect</code>
<code>triangle(...)</code> , <code>quad(...)</code>	<code>triangle</code> , <code>quad</code>	closed polygons
<code>polyline(points)</code>	<code>beginShape ... vertex ... endShape</code> , <code>inside</code> <code>push / noFill / pop</code>	<code>polyline</code>
<code>polygon(points)</code>	the same, with <code>endShape(CLOSE)</code>	<code>polygon</code>

Points for `polyline` and `polygon` may be `[x, y]` pairs or `{ x, y }` objects; both are common in p5 code. Every shape method returns the plot, so calls can be chained. The choices here follow the engine: it flattens a circle to 96 segments, so an oval gets the same; its stipple already drew a quarter-millimetre dash, so a point does too.

6 FILLS

p5 fills a shape with colour; a pen cannot. The three fill methods hand a polygon to the engine's fills, in millimetres, and draw the resulting lines back onto the canvas with the sketch's current stroke, so screen and paper show the same hatching:

```
let square = [[20, 80], [120, 80], [120, 180], [20, 180]];
plot.hatch(square, 3, PI / 4);    // spacing 3 mm on the bed, angle in angleMode()
plot.crossHatch(square, 4);      // second pass at right angles
plot.stipple(square, 150, 3);    // 150 dots, seed 3: same seed, same dots
```

The spacing is a physical distance, not a pixel count: 3 mm reads as a mid grey whatever the canvas size. The angle follows `angleMode()` like every other angle in the sketch. Stipple dots come back as `point()` calls at the centre of each dash. The engine's own optimizer later joins the hatch lines end to end and orders them, which is why a hatched square plots as a zigzag and not as a hundred separate strokes.

7 THE FRAME MODEL: SIXTY FRAMES, ONE PLOT

`draw()` runs sixty times a second; a plotter draws once. The adapter does not try to reconcile the two. It records whatever the shape methods were called with since the last `clear()`, and `go()` plots that. So what gets plotted is always the last frame you saw, which is the natural thing: you look, you like it, you click it.

```
function draw() {  
  background(255); // screen only: no plot. in front of it, so it is not recorded  
  plot.clear();    // new frame, new job  
  plot.rect(10, 10, 380, 230);  
  plot.line(30, 125, 370, 125);  
  plot.go();       // ready: click the drawing to plot  
}
```

Two habits make this comfortable. Call `noLoop()` in `setup()` and `redraw()` on a key when the sketch is generative, so a plot never races a frame; the Molnár grid does this with R for a new seed. And keep `randomSeed()` at the top of `draw()`, so the frame you plotted can be drawn again tomorrow. A sketch that animates can still be plotted: the frame that is on screen when you click is the one that goes to paper.

8 PLOT.GO(): CONNECT, CONFIRM, PLOT, STOP

`go()` is one method because a person at a plotter wants one gesture: you write it at the end of the drawing, and a click on the drawing starts the pen. Called from `setup()` or `draw()`, where there is no click yet, it writes the safety checklist to the status line and waits for one click on the canvas; that click is the confirmation, and no dialog follows. Called from a key or mouse handler, it goes straight on and asks once with a dialog. A second `go()` while one is waiting does not queue a second plot. Inside it does four things in order:

1. **Require the driver.** Without one installed it throws: "plot.go() needs the driver: `installP5Penplotter(p5, PlotterEngine, { driver: Ebb })`." If a plot is already running it returns `{ status: "busy" }` and does nothing.
2. **Plan.** `plan()` on the engine, which optimises and orders the frame's geometry.
3. **Connect.** `connect()` reuses an open transport; otherwise it takes a port the browser already granted (`navigator.serial.getPorts()`) or opens the browser's port picker with no USB filter, so an EBB clone with unusual ids still shows up; the driver's `v` handshake then decides. A closed picker becomes a plain message: "No plotter was chosen. If no list appeared at all, open this page in Chrome or Edge itself."
4. **Confirm and run.** The `confirm` function (a dialog by default) asks whether the carriage is parked in the home corner, the paper is in place with no magnet in the way, and hands are clear. A refusal returns `{ status: "cancelled" }` and sends nothing. Otherwise `driver.run(plan, { confirmed: true })` streams the plan; the result is returned, and the status line reads "plot complete" or "stopped safely: ..." on an error, which is then rethrown so the sketch can show it.

While the pen runs, a click on the drawing calls `stop()` ; so does `plot.stop()` from your own code. It calls the driver's `abort()`, which ends in the engine's safe stop: motion cleared, pen up, motors off. Every status change goes through `say()` : to the `log` function when you gave one, otherwise to the console. The examples pass a function that writes into a status line on the page.

The confirmation is the adapter's own, on top of the engine's `confirmed: true` requirement; the bounds check, the speeds, the flow control and the safe stop are the engine's and are not duplicated here. The engine's handbook, chapter 9, describes what happens on the wire.

9 CREATEPLOTTERENGINE() AND DRAWROUTE()

`createPlotterEngine(options)` returns a bare `PlotterEngine` whose page is the current canvas: `width` and `height` from the sketch, `margin` 0, and the unit `px` unless you say otherwise. It is for sketches that want the engine's full surface, layers, pens, the optimizer's options, the exports, without the draw-and-record object. Because its unit is pixels, such an engine can preview and export (the exports convert pixels to physical sizes at 96 per inch) but cannot plot directly; give it `units: "mm"` and millimetre coordinates for that, or use `createPlot()`.

`drawRoute(plan, options)` draws a plan's routes with p5: `push`, `noFill`, then for every route a `stroke` in its tool's colour (or `options.stroke`), `strokeWeight(options.strokeWeight ?? 1)`, and a `beginShape / vertex / endShape ; pop` at the end. It draws `plan.routes`, the paths in the order the machine will draw them, never the original artwork, so what you see is the planner's result including reversed paths and moved seams. It refuses anything that is not a `vanilla.penplotter/plan@1` with a routes array.

`drawPlanWithP5(p, plan, options)` is the same function without the prototype helper, for code that has a p5 instance in hand.

10 WHAT IS NOT RECORDED

The adapter records the arguments you pass to its shape methods. It does not look at p5's state, and that is the rule to remember when the paper differs from the screen:

- **Transforms.** `translate()`, `rotate()` and `scale()` move what the canvas shows, not what is recorded. A shape drawn inside a transform lands on paper untransformed. Compute the transformed coordinates yourself and pass those, or stay in absolute coordinates as the examples do.
- **rectMode and ellipseMode.** The canvas honours them; the engine gets corner-mode rectangles and centre-mode ellipses, p5's defaults. Keep the defaults, or convert.
- **Stroke weight and colour.** The pen is the pen. Everything is recorded on the engine's default tool; the SVG export has one hairline and the plot has whatever pen is in the holder. For two pens, work on `plot.engine` directly with `tool()` and `layer()`.
- **Fills.** `fill()` is a screen thing. Use `hatch`, `crossHatch` or `stipple`.
- **Everything without `plot.`**, which is the point: `background()`, a debug text, a mouse cursor, a grid you draw to help yourself, all stay on screen.

11 P5.WAVES IN A SKETCH

`Waves.wave(x, options)` returns a number; the adapter never sees the library. Build ordinary points with it and pass them to `polyline`. Two things matter for paper: pick a fixed `t`, because the plotter draws one moment, and use the wave for something a pen can show, such as the y of a line, the angle of a field, or the spacing of a hatch.

```
const points = [];  
for (let x = 40; x <= 760; x += 4) {  
  points.push({ x, y: 400 + Waves.wave(x, { wave: "triangle sine", t: 0, amplitude: 80,  
frequency: 0.04 }) });  
}  
plot.polyline(points);
```

The Wave field example goes one step further: two waves, one per axis, give every point of the sheet a direction, and sixty lines follow that direction until they leave the frame. The seed picks the wave shape, so one sketch is thirty-four different fields.

12 THE EXAMPLES, ONE BY ONE

EXAMPLE	WHAT IT SHOWS	WIRING
<code>direct_plot</code>	five wave lines in a frame; a click on the drawing plots, a second click stops; the status line is the adapter's log	instance mode, sibling imports
<code>first_plot</code>	<code>createPlotterEngine()</code> and <code>drawRoute()</code> : a hatched polygon and a circle, drawn as the planner ordered them	instance mode
<code>wave_plot</code>	24 rows sampled from one of p5.waves' 34 formulas; a button rerolls the wave	instance mode
<code>molnar_grid</code>	ten rows of nested squares that drift and turn a little more each row; one seed, R rerolls; a plain global-mode sketch with no imports	global mode, wiring in <code>index.html</code>
<code>calibration_sheet</code>	a 100 mm ruler, hatch and cross-hatch scales down to 0.8 mm, stipple densities, circles from 2 to 40 mm, line pairs from 0.3 mm apart: only physical variables; plot it once per pen	global mode
<code>wave_field</code>	streamlines through a direction field shaped by p5.waves; thirty-four fields in one sketch	global mode
<code>spirograph</code>	three hypotrochoids around one centre, each a single unbroken polyline, wheels chosen so every curve closes within eleven turns	global mode
<code>chaos_game</code>	one point jumps halfway to a random corner and leaves a dot, 1 500 times; <code>plan({ strategy: "drawn" })</code> keeps the order of the game, so the Sierpinski triangle appears on paper dot by dot, half an hour long	global mode

The global-mode examples are the test of the root: a `sketch.js` with no imports, no instance mode, only `createPlot`, `polygon` with `[x, y]` pairs, `random` and a for-loop, with the wiring in `index.html` as Setup 1 describes it. Every example page accepts `?embed`, which hides the navigation and the panel so the gallery can show the live canvas in a frame.

13 RECIPES

RINGS

```
function draw() {  
  background(255); noFill();  
  plot.clear();  
  for (let d = 40; d <= 560; d += 20) plot.circle(300, 300, d);  
}
```

A FRAME YOU CAN MEASURE

```
// 300 px wide canvas, 300 mm on the bed: one pixel is one millimetre, so a 40 px square is  
40 mm on paper  
plot = createPlot({ x: 147, y: 66, width: 300 });  
plot.square(20, 20, 40);
```

TWO PENS FROM A SKETCH

```
const engine = plot.engine; // the vanilla.penplotter engine behind the plot  
engine.tool({ id: "red", color: "#c8402c", width: 0.8 });  
engine.layer("accent", { toolId: "red" }); // recorded shapes now land on this layer  
plot.circle(300, 300, 200);  
engine.layer("default"); // back to the black pen  
plot.rect(20, 20, 560, 560);  
// plot.go() pauses once, after the default layer, and asks for the red pen
```

A SERIES

```
function draw() {  
  randomSeed(seed);  
  // ... the drawing ...  
  plot.go(); // like it? click it  
}  
  
function keyPressed() {  
  if (key === "r") { seed += 1; redraw(); } // look again  
}
```

PLOT A TRANSFORMED SHAPE CORRECTLY

```
// wrong: the rotation is on the canvas only
// rotate(a); plot.rect(0, 0, 100, 40);
// right: rotate the corners yourself and pass a polygon
const corners = [[0, 0], [100, 0], [100, 40], [0, 40]].map(([x, y]) => [
  cx + x * cos(a) - y * sin(a), cy + x * sin(a) + y * cos(a)
]);
plot.polygon(corners);
```

14 PITFALLS, WITH THEIR ERROR MESSAGES

WHAT YOU SEE	WHAT HAPPENED	FIX
<code>createPlot</code> is not defined	<code>sketch.js</code> is a module, or <code>createPlot()</code> is called at the top of the file before the adapter is installed	plain script; call it inside <code>setup()</code>
<code>p5</code> is not defined in the module	the p5 script tag comes after the module, or failed to load	load p5 first
"needs vanilla.penplotter 0.2.0 or newer"	an old engine, often the browser's cached copy after an update	reload with Ctrl+F5, or update the pinned tag
"plot.go() needs the driver"	installed without <pre>{ driver: Ebb }</pre>	pass the engine's <code>src/driver/ebb.js</code> module
"No plotter was chosen. If no list appeared at all ..."	no port picker: not Chrome or Edge, not localhost or https, or an iframe (the Web Editor's preview)	open the page in Chrome or Edge itself, from a local server, and plot from a key
"Point (...) mm is outside the machine travel"	the canvas at its offset and width does not fit the 594 × 432 mm bed	smaller <code>width</code> or a smaller offset
Cannot redefine property: <code>square</code>	the sketch defines a function with the name of a p5 global (<code>square</code> , <code>circle</code> , <code>point</code> , ...)	rename it; p5 2.x protects its globals
the paper differs from the screen	a transform, <code>rectMode</code> , <code>ellipseMode</code> or a fill that the canvas	chapter 10

WHAT YOU SEE	WHAT HAPPENED	FIX
	honoured and the recording did not	
the plot is the previous frame	<code>go()</code> plots what was recorded since the last <code>clear()</code>	call <code>plot.clear()</code> at the top of <code>draw()</code>

15 TESTS, TOOLING AND THE SITE

`npm test` runs three Node scripts. `tests/snapshot.js` installs the adapter into a fake p5 and a fake engine, checks that `drawPlanWithP5` makes exactly the expected p5 calls, walks every local link on the site, and checks that every example is in the gallery and the manifest and that the generated architecture page is current. `tests/plot.js` checks every shape method against the fake engine, the degrees conversion, the scale and offset, `go()`'s flow with a fake driver kit (connection reuse, cancellation, the missing-driver error), and then, when the engine's checkout is next door, the same against the real engine and its EBB driver: 80 pixels at 200 mm per 400 pixels is 40 mm is 3 200 steps on both motors, hatch lines drawn back equal the paths planned, the same seed gives the same dots, and a canvas placed off the bed is refused. `tests/requirements.js` checks the version contract everywhere it is stated and compares the shared table with the engine's README.

`npm run docs` renders the architecture page from Markdown, `npm run manifest` regenerates the manifest, `npm run handbook` prints this page to PDF. The GitHub Actions workflow checks out the engine next to the adapter, because both the examples and the tests use it as a sibling folder, and fails when a generated file is stale. Publishing order: the engine first, then the adapter.

16 LIMITS AND ROADMAP

- No curves: `bezier`, `curve` and `curveVertex` are not wrapped yet. Sample them into points, or draw them with `polyline`.
- No transforms, modes, stroke weights or fills, as chapter 10 describes.
- No `layer()` or `tool()` on the plot itself; two pens go through `plot.engine`.
- One machine profile, one browser family, one click: the engine's limits are the adapter's.

Next on the roadmap for the adapter: arcs and curves as p5 primitives, a pen and layer convenience on the plot, and, when the engine has it, pause and resume. Everything about speed, acceleration and machines belongs to the engine and is listed in its handbook.

17 REFERENCE

EXPORT	WHAT IT IS
<code>installP5Penplotter(p5, PlotterEngine, { driver })</code>	installs the three methods on <code>p5.prototype</code> ; returns the constructor
<code>REQUIRES</code>	<code>{ core: "0.2.0", p5: "2.2.2" }</code> , the contract every other statement is tested against
<code>P5Plot</code>	the class behind <code>createPlot()</code> : <code>clear</code> , <code>toMm</code> , <code>toPx</code> , the shape and fill methods, <code>plan</code> , <code>connect</code> , <code>go</code> , <code>stop</code> , <code>say</code> ; fields <code>engine</code> , <code>scale</code> , <code>offset</code> , <code>status</code> , <code>busy</code>
<code>drawPlanWithP5(p, plan, options)</code>	the renderer behind <code>drawRoute()</code>
<code>P5Penplotter</code> (default export)	<code>{ version, requires, install, draw }</code>

CREATEPLOT OPTION	DEFAULT
<code>x</code> , <code>y</code>	0, 0 mm from home
<code>width</code> or <code>mmPerPixel</code>	1 mm per pixel
<code>profile</code>	<code>idraw-hse-a2</code>
<code>confirm</code> , <code>log</code>	<code>window.confirm</code> , <code>console.log</code>

GO() RESULT	MEANING
<code>{ status: "complete", commands, durationMs }</code>	the driver sent everything
<code>{ status: "aborted", index }</code>	<code>stop()</code> was called; the machine ended in the safe stop
<code>{ status: "cancelled" }</code>	the confirmation was declined; nothing was sent
<code>{ status: "busy" }</code>	a plot was already running
a thrown error	no driver, no port chosen, a point off the bed, or a machine error; the status line says which

This handbook describes version 0.2.1 of 2026-10-02. Where the code and this text disagree, the code and its tests are right, and the text needs a fix: open an issue on GitHub.

